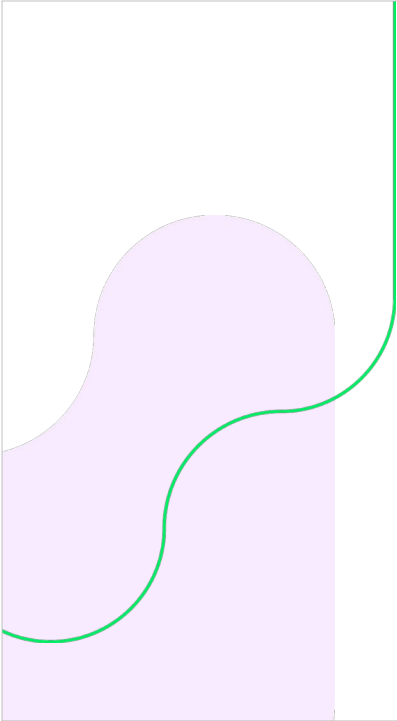




DS110

Atlas Hybrid Search

Atlas Search and Vector Search



Topics we cover

Hybrid Search

Combined Queries

Fusion Algorithms

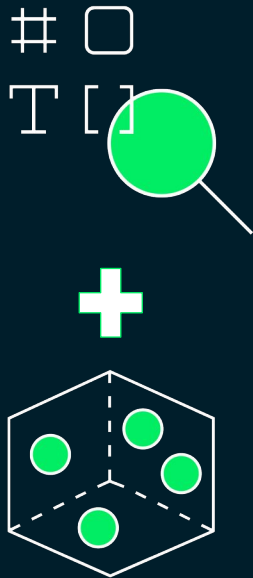
Relative Score (RS)

Reciprocal Rank (RR)

Hybrid Search



What is Hybrid Search?



Full text search:

lexical/token search

Vector Search:

semantic search

Hybrid Search:

Text + Vector Search

You can leverage the capabilities of both full text and semantic search within Atlas to improve the accuracy and performance of your search application.

Hybrid Search is a feature that combines traditional full text search and semantic search to produce comprehensive and contextually relevant search results.



Why Hybrid Search?

fighting ghosts bill murray

Show Indexes Sample Doc

Fulltext Search Vector Search Relative Score Fusion Reciprocal Rank Fusion

query took 13ms Show Query

Fighting

In New York City, a young counterfeiter is introduced to the world of underground street fighting by a seasoned scam artist, who becomes his manager on the bare-knuckling brawling circuit.

cast : Channing Tatum, Terrence Howard, Zuyi Henao, Michael Rivera

genres : Action, Drama, Sport

Wild Bill

The early career of legendary lawman Wild Bill Hickock is telescoped and culminates in his relocation in Deadwood and a reunion with Calamity Jane.

cast : Jeff Bridges, Ellen Barkin, John Hurt, Diane Lane

genres : Action, Biography, Western

Ghosts of Girlfriends Past

While attending his brother's wedding, a serial womanizer is haunted by the ghosts of his past girlfriends.

cast : Matthew McConaughey, Jennifer Garner, Michael Douglas, Breckin Meyer

genres : Comedy, Fantasy, Romance

Text Search

6

Why use Hybrid Search if we already have vector and text search?
Sometimes the results from these searches are relevant, but they are not ranked optimally.
If we combine both search types, we can obtain more relevant results.

Let's look at an example with the movies collection. When searching "fighting ghosts bill murray" using full text search, the top results match the keywords in the query, but they are not the most relevant.

Optional Instructor live demo: <https://hybrid.sa.prod.corp.mongodb.com/>



Why Hybrid Search?

fighting ghosts bill murray

Show Indexes Sample Doc

Fulltext Search **Vector Search** Relative Score Fusion Reciprocal Rank Fusion

numCandidates: 100, query took 22ms

Show Query

A Christmas Carol: The Musical
An old bitter miser is given a chance for redemption when he is haunted by ghosts on Christmas Eve.

Casper
A paranormal expert and his daughter bunk in an abandoned house populated by 3 mischievous ghosts and one friendly one.

Ghost Graduation
A teacher with paranormal abilities helps a group of ghosts graduate high school.

Yellow Sky
Pistol-packing tomboy, and grandfather come to discover band of bank robbing bandits taking refuge in the neighboring ghost town.

Vector Search

7

If we run a semantic search using the same query **"fighting ghosts bill murray"** (based on the movie plot meaning), we get additional results that could be relevant, but don't rank as the most relevant for this example.

Optional Instructor live demo: <https://hybrid.sa.prod.corp.mongodb.com/>



Why Hybrid Search?

Vector Search

Show IndexesSample Doc

Fulltext SearchVector SearchRelative Score FusionReciprocal Rank Fusion

numCandidates: 100, query took 42msShow Query

Ghostbusters II



The discovery of a massive river of ectoplasm and a resurgence of spectral activity allows the staff of Ghostbusters to revive the business.

vs_score : 0.6406923109367008

fts_score : 0.09895712050369315

cast : Bill Murray, Dan Aykroyd, Sigourney Weaver, Harold Ramis

genres : Comedy, Fantasy

Scoring for this result: $0.099 + 0.641 = 0.740$

Lexical 13% 87% Vector

Ghost Graduation



A teacher with paranormal abilities helps a group of ghosts graduate high school.

If we combine both searches using the same query “**fighting ghosts bill murray**”, our results are more likely to be relevant
There are different ranking implications when we calculate this hybrid search, we will discuss them in detail in this lesson.

Optional Instructor live demo: <https://hybrid.sa.prod.corp.mongodb.com/>



When to use Hybrid Search?

Vector Search matches on semantic meaning

Dependant on a Large Language Model (LLM)

LLM might not have been trained on our data/keywords

-> combine with lexical (keyword) search

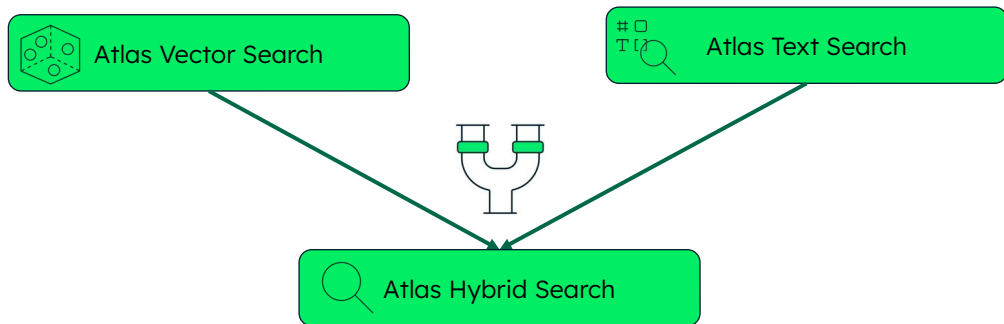
Action movies starring Arnold Schwarzenegger 

Semantic search (Vector search) is very powerful but can sometimes come up short if the embedding model was trained on sample data that didn't include the keywords we are using for the search.

The vectors the embedding model produced wouldn't be well suited to finding what we're looking for. In this case, full text search (Lexical search) is better equipped to find the keywords.



How to apply Hybrid Search?



10

The most common approach is to execute two queries, a text query and a vector query, and then combine the results from each into a new ranked set

There is a way to fuse the results from both, Vector search queries and Atlas full-text search queries, to determine which result is best.

Applying rank algorithms to the results from both queries we can sort the final results combining both.

Combined Queries





Combining both search queries

\$unionWith aggregation stage **combines** results from multiple aggregations:

```
db.collection1.aggregate([
  { $project: { field1: 1, field2: 1, _id: 0 } },
  { $unionWith:
  { coll: "collection2",
    pipeline: [ { $project:{ field1: 1,field2: 1, _id: 0 } } ]}
  }
])
```

How many documents will this operation return?

field1	field2
A	100
B	150



field1	field2
C	50
B	150
E	20

In order to run the hybrid search, we need to remember who the aggregation command and aggregation stages work.

MongoDB \$unionWith aggregation stage allow us to combine 2 collections and supports \$search inside the \$unionWith pipeline option

the \$unionWith stage is working as a UNION ALL in SQL, so based on the example, the operation will return 5 documents (includes duplicates {field1:"B", field2:"150"})



Combining both search queries

\$group aggregation stage **removes duplicates** from results:

```
db.collection1.aggregate([
  { $project: { field1: 1, field2: 1, _id: 0 } },
  { $unionWith:
  { coll: "collection2",
    pipeline: [ { $project:{ field1: 1, field2: 1, _id: 0 } } ] }
  },
  { $group: { _id: "$field1" } }
])
```

We can add other aggregation stages (\$sort) to define sort order of the results

field1	field2
A	100
B	150



field1	field2
C	50
B	150
E	20

In order to run the hybrid search, we need to remember who the aggregation command and aggregation stages work.

MongoDB \$unionWith aggregation stage allow us to combine 2 collections and supports \$search inside the \$unionWith pipeline option

the \$unionWith stage is working as a UNION ALL in SQL, so based on the example, the operation will return 5 documents (includes duplicates {field1:"B", field2:"150"})



Combining both search queries

Example using \$vectorSearch and \$search

```
db.embedded_movies.aggregate([
  { $vectorSearch: { "index": 1, "path": 1, "queryVector": [...], "numCandidates": 100, "limit": 20 } },
  { $project: { title: 1, plot: 1 } },
  { $unionWith:
{ coll: "embedded_movies",
  pipeline: [
    { $search: {index: 'default', text:{ query:"user search", "path": ["title","plot", ...] } } },

{ $project: { title: 1, plot: 1 , ...} } ]} },
    { $group:{ _id: "$title" } , ...}
  ]})
```

14

\$vectorSearch must be the first stage of any pipeline where it appears.

When using \$unionWith we can use the \$search stage in the union pipeline and use \$group for group per movie title.

Questions to discuss:

- What else you need to add to the \$group stage?
- Are there other stages you need to include after \$group?



Exercise

Let's create a simple example using the ***sample_mflix.embedded_movies*** collection.

This exercise follows these steps to understand the process:

Full Text Search: Using Atlas Search with an Atlas Search index, find movies containing “x” in their title, plot, cast, or genres

Vector Search: Using Vector Search with an Atlas Vector Search index, find movies with a similar meaning to “x” in their plot

Hybrid Search: Combine results from 1st and 2nd steps using a Hybrid Search
Our search query (x) will be the following:

capturing ghosts bill murray



15

Let's analyze the results of the different searches based on the search query:

capturing ghosts bill murray

Make sure you have access to an Atlas Cluster (M0 +) with the Sample Dataset loaded

Exercise - Part I: Full Text search



Retrieve movies with the keywords “*capturing ghosts bill murray*”.

Search for these keywords in title, plot, genre and cast fields.

Connect to your Atlas cluster

Create an Atlas Search Index (called *default*)

Database	Collection	Index Name	Type	Index Fields	Status
sample_mflix	embedded_movies	default	search	"cast", "genres", "plot", "title"	READY

Run the aggregation query using the `$search`

```
{
  "mappings": {
    "dynamic": false,
    "fields": {
      "cast": {
        "analyzer": "lucene.standard",
        "multi": {
          "keywordAnalyzer": {
            "analyzer": "lucene.keyword",
            "type": "string"
          },
          "type": "string"
        },
        "type": "string"
      },
      "genres": [
        {
          "type": "stringFacet",
          "type": "token",
          "multi": {
            "keywordAnalyzer": {
              "analyzer": "lucene.keyword",
              "type": "string"
            },
            "type": "string"
          },
          "type": "string"
        },
        {
          "type": "string",
          "type": "token",
          "multi": {
            "keywordAnalyzer": {
              "analyzer": "lucene.keyword",
              "type": "string"
            },
            "type": "string"
          },
          "type": "string"
        }
      ],
      "plot": {
        "type": "string",
        "type": [
          {
            "type": "string",
            "type": "autocomplete"
          }
        ]
      }
    }
  }
}
```

16

Create or modify your existing **text search index**, called **default**, and use the JSON definition from the slide.

Once the index is created you can run this search from your terminal:

```
let query = [
  {
    $search: {
      index: 'default',
      text: {
        query: "capturing ghosts bill murray",
        path: ["title", "plot", "cast", "genres"],
        "value": "cast",
        "multi": "keywordAnalyzer"
      },
      "value": "genres",
      "multi": "keywordAnalyzer"
    }
  },
  {
    $project: {
      title: 1,
      plot: 1,
      poster: 1,
      cast: 1,
      genres: 1,
      rating: "$imdb.rating",
      score: {
        $meta: "searchScore"
      }
    }
  },
  {
    $sort: {
      score: -1
    },
    $limit: 10
  }
]
```

`db.embedded_movies.aggregate(query)`

Alternatively, you can run this search using the **movies_recommendations_hybrid** python project to test this (available in your lab IDE)

In the **params.py** include the Atlas connection string and the API token (your instructor will provide you with an OpenAI Key)

In the **recommend.py** script, review the **fulltextSearch()** function and the pipeline variable

In the Terminal 2 tab execute these commands:

```
cd movie_recommendations_hybrid
python3 -m venv env
source env/bin/activate
pip3 install -r requirements.txt
```

Test in the Terminal 2 tab:

```
python3 recommend.py -t "capturing ghosts bill murray" -s "text"
```

Expected Output - 10 documents

Exercise - Part I: Full Text (Results)



Search Results

	Title	Search Score
	Wild Bill - Plot: The early career of legendary lawman Wild Bill Hickock is telescoped and culminates in his relocation in Deadwood and a reunion with Calamity Jane. - Cast: ['Jeff Bridges', 'Ellen Barkin', 'John Hurt', 'Diane Lane'] - Genres: ['Action', 'Biography', 'Western'] - Rating: 5.9	Score: 6.197145462036133
	Ghosts of Girlfriends Past - Plot: While attending his brother's wedding, a serial womanizer is haunted by the ghosts of his past girlfriends. - Cast: ['Matthew McConaughey', 'Jennifer Garner', 'Michael Douglas', 'Breckin Meyer'] - Genres: ['Comedy'] - Rating: 5.7	Score: 5.597512245178223
	Bill & Ted's Bogus Journey - Plot: A tyrant from the future attempts to destroy the world. - Cast: ['Keanu Reeves', 'Alex Winter', 'Anthony Michael Hall'] - Genres: ['Adventure', 'Comedy', 'Science Fiction'] - Rating: 6.2	
	Kill Bill: Vol. 2 - Plot: The Bride - Cast: ['Vivica A. Fox', 'Lucy Liu', 'Michael Madsen'] - Genre: 'Action'	
	Ghostbusters - Plot: Three unemployed parapsychology professors set up shop as a unique ghost removal service. - Cast: ['Bill Murray', 'Dan Aykroyd', 'Sigourney Weaver', 'Harold Ramis'] - Genres: ['Comedy', 'Fantasy'] - Rating: 7.8	Score: 4.48095703125
	Ghostbusters II - Plot: The discovery of a massive river of ectoplasm and a resurgence of spectral activity allows the staff of Ghostbusters to revive the business. - Cast: ['Bill Murray', 'Dan Aykroyd', 'Sigourney Weaver', 'Harold Ramis'] - Genres: ['Comedy', 'Fantasy'] - Rating: 6.5	Score: 4.48095703125

Full text search results

Ghostbusters movies appear on 7th & 8th position

searchScore: 4.4809...

You can run this search using the **movies_recommendations_hybrid** python project to see these results:

```
python3 app.py
```

NOTE: Attempting to run the command `python3 app.py` within the VSCode integrated terminal will cause the Instruqt instance to crash. Use the bash terminal tab instead.

From the browser, use the UI (website) to search:

- Input the search: `capturing ghosts bill murray`
- Use the **FullText search** button



Exercise - Part II: Vector search

Retrieve movies where their **plot** is *semantically* related to “*capturing ghosts bill murray*”
 Create an Atlas Vector Search Index if needed (called *vector_index*) using euclidean

```
{
  "fields": [
    {
      "numDimensions": 1536,
      "path": "plot_embedding",
      "similarity": "euclidean",
      "type": "vector"
    }
  ]
}
```

vector_index

vectorSearch

plot_embedding

ACTIVE

[View status details](#)

Use the OpenAI text-embedding-ada-002 model to generate the vector embedding
 Run a semantic search using \$vectorSearch to find movies with plot similarities to the query.
*Note: Use the **movies_recommendations_hybrid** python project to test this*

18

Use the **movie_recommendations_hybrid** project (available in your lab IDE)
 In the **params.py** include the connection string and the API token for our OpenAI azure resource (your instructor will provide you with an OpenAI Key)
 In the **recommend.py** script, review the **vectorSearch()** function and the pipeline variable

```
pipeline = [
  {"$vectorSearch": {
    "index": "vector_index",
    "queryVector": embedding,
    "path": "plot_embedding",
    "limit": 10, # number of nearest neighbors to return
    "numCandidates": 10 # number of HNSW entry points to explore
  }},
  {"$project": { "_id": 0, "title": 1, "plot": 1,
    "rating": "$imdb.rating", "score": { '$meta': "vectorSearchScore" } }},
  {"$limit": 10 }
]
```

In a new VSCode terminal execute these commands:

```
cd movie_recommendations_hybrid
python3 -m venv env
source env/bin/activate
pip3 install -r requirements.txt
```

Test in the VSCode Terminal:

```
python3 recommend.py -t "capturing ghosts bill murray" -s "vector"
```

Exercise - Part I: Vector (Results)



Title		Search Score
	Ghostbusters II - Plot: The discovery of a massive river of ectoplasm and a resurgence of spectral activity allows the staff of Ghostbusters to revive the business. - Cast: [Bill Murray, Dan Aykroyd, Sigourney Weaver, Harold Ramis] - Genres: [Comedy, Fantasy] - Rating: 6.5	Score: 0.7215888500213623
	Casper - Plot: A paranormal expert and his daughter bunk in an abandoned house populated by 3 mischievous ghosts and one friendly one. - Cast: [Chauncey Leopardi, Spencer Vrooman, Malachi Pearson, Cathy Moriarty] - Genres: [Comedy, Family, Fantasy] - Rating: 6.0	Score: 0.7201488614082336
	Beetlejuice - Plot: A couple of recently deceased ghosts contract the services of a "bio-exorcist" in order to remove the obnoxious new owners of their house. - Cast: [Alec Baldwin, Geena Davis, Annie McEnroe, Maurice Page] - Genres: [Comedy, Fantasy] - Rating: 7.4	Score: 0.7169212698936462
	Chasing Christmas - Plot: In a world where the holidays are run by the efficient Bureau of Yuletide Affairs, a bitter and burned-out Ghost of Christmas Past decides to go AWOL while on a "mission" and leave his "target" stranded in 1965. - Cast: [Tom Arnold, Andrea Roth, Leslie Jordan, Brittney Wilson] - Genres: [Comedy, Fantasy] - Rating: 5.4	Score: 0.7166777849197388
	Ghostbusters - Plot: Three unemployed parapsychology professors set up shop as a unique ghost removal service. - Cast: [Bill Murray, Dan Aykroyd, Sigourney Weaver, Harold Ramis] - Genres: [Comedy, Fantasy] - Rating: 7.8	Score: 0.7133632302284241

Vector search results

With the search parameters as follows:

- limit= 10
- numCandidates=10

Ghostbusters movies appear on the 1st and 5th position

vectorSearchScore:
0.7215..... / 0.7133...

You can run this search using the **movies_recommendations_hybrid** python project to see these results:

```
python3 app.py
```

From the browser, use the UI(website) to search:

- Input the search: **capturing ghosts bill murray**
- Use the **Vector search** button

Note: Vector search only is taking in consideration the movie plot and search on meaning similarities. Also note the score in vector search is using a different scale (where 1 is the exact match) while the Full text search score is much higher.

Remember the search parameters like numCandidates and limit can also influence the result rankings of the vector search



Exercise - Part III: Hybrid

Run the same query as before but using the combination of both using `$unionWith` and `$group`:

```
pipeline=[
{"$vectorSearch": . . .} ,
{"$project": . . ., "vs_score": { '$meta': "vectorSearchScore" }} ,
{"$limit": . . .} ,
{"$unionWith":
  { "coll": "embedded_movies",
    "pipeline": [
      { "$search": { "index": 'default', "text":{ "query":text, "path": [...] } } },
      { "$project": . . ., "ts_score": { "$meta": "searchScore" } }
    ] } },
{"$group":{ "_id": "$title" } , . . .}
{"$addFields":{ "score": { "$add: ["ts_score", "vs_score"]}}}
{"$sort":{ "score": -1 } }
{"$limit":10}
{"$project": . . .,}
]
```

Use the `movie_recommendations_hybrid` project, review the hybrid search and the results

20

Use the **movie_recommendations_hybrid** project (available in your lab IDE)

In the **params.py** include the connection string and the API token for our OpenAI azure resource (your instructor will provide you with an OpenAI Key)

In the **recommend.py** script, review the **hybridSearch()** function and the pipeline variable

Note:

- We run vector search and then the union with the full text search results. After that we group the results and project
- When we run each `$vectorSearch` and `$search` stages we store the score from each search in separate field (`ts_score` and `vs_score`) - we have seen these scores are different so we will need to apply some logic to use them together.
- For the final score, we are adding the `ts_score` and `vs_score` together

In a new VSCode terminal execute these commands:

```
cd movie_recommendations_hybrid
python3 -m venv env
source env/bin/activate
pip3 install -r requirements.txt
```

Test in the VSCode Terminal:

```
python3 recommend.py -t "capturing ghosts bill murray" -s "hybrid"
```

The results seems the same as the full text search

Could you think of a reason why this is not the correct approach?

Exercise - Part III: Hybrid (Results)



Wild Bill

• Plot: The early career of legendary lawman Wild Bill Hickock is telescoped and culminates in his relocation in Deadwood and a reunion with Calamity Jane.

• Cast: [Jeff Bridges', 'Ellen Barkin', 'John Hurt', 'Diane Lane']

• Genres: [Action', 'Biography', 'Western']

• Rating: 5.9

Score:
6.197145462036133



Ghosts of Girlfriends Past

• Plot: While attending his brother's wedding, a serial womanizer is haunted by the ghosts of his past girlfriends.

• Cast: [Matthew McConaughey', 'Jennifer Garner', 'Michael Douglas', 'Breckin Meyer']

• Genres: [Comedy', 'Fantasy', 'Romance']

• Rating: 5.7

Score:
5.597512245178223



Bill & Ted's Bogus Journey

• Plot: A tyrant from the future creates evil android doubles of Bill and Ted and sends them back to eliminate the originals.

• Cast: [Keanu Reeves', 'Alex Winter', 'William Sadler', 'Joss Ackland']

• Genres: [Adventure', 'Comedy', 'Fantasy']

• Rating: 6.2

Score:
5.514348983764648



Kill Bill: Vol. 2

• Plot: The Bride continues her quest of vengeance against her former boss and lover Bill, the reclusive bouncer Budd and the treacherous, one-eyed Elle.

• Cast: [Vivica A. Fox', 'Ambrosia Kelley', 'Michael Parks', 'James Parks']

• Genres: [Action', 'Crime', 'Thriller']

• Rating: 8.0

Score:
5.372238159179687



Ghostbusters II

• Plot: The discovery of a massive river of ectoplasm and a resurgence of spectral activity allows the staff of Ghostbusters to revive the business.

• Cast: [Bill Murray', 'Dan Aykroyd', 'Sigourney Weaver', 'Harold Ramis']

• Genres: [Comedy', 'Fantasy']

• Rating: 6.6

Score:
5.202545881271362

Hybrid Search results

Ghostbusters movies
appear on the 5th & 6th
position

score:

5.202... / 5.1...

INCORRECT

21

You can run this search using the **movies_recommendations_hybrid** python project to see these results:

`python3 app.py`

From the browser, use the UI(website) to search:

- Input the search: **capturing ghosts bill murray**
- Use the **Hybrid Search** button

The issue is the final "score" we use to sort the results, the text search scores where by default higher than the vector scores, so by adding both we are not getting the correct rankings for combining this results by relevance.



Getting the right scoring

When working with Hybrid Search we need to get the final score (combined score) correct.



```
"ts_score": { "$meta": "searchScore" }
```



```
"vs_score": { "$meta": "vectorSearchScore" }
```



```
{ "$addFields":  
  { "score":  
    { "$add":  
      [ "ts_score", "vs_score" ]  
    }  
  }  
}
```

ts_score and vs_score are not on the same scale, so we cannot just sum these 2 values

While vector search scores upper limit is 1, text search scores can go over 1, so just by adding both values, we will be giving more relevance to the text search results.

Let's review fusion algorithms to make this scoring more relevant to both searches.

Fusion Algorithms





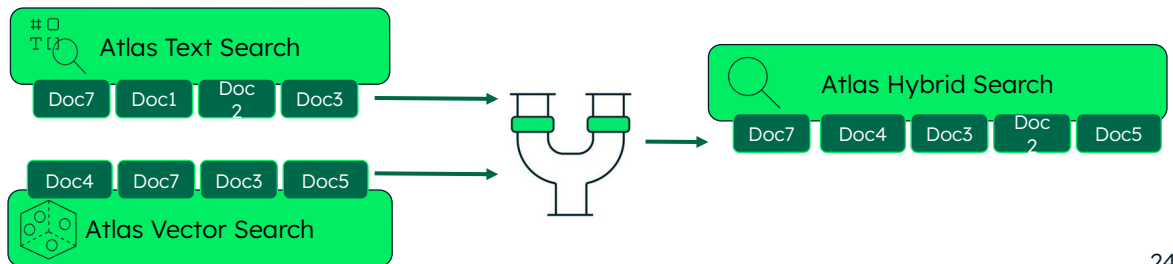
Fusion algorithms

Sorting the combined search results using a new rank/ combined score is key.

The fusion algorithms re-rank or re-score the documents in the combined collection:

Relative Score Fusion (RSF): based on normalized scores

Reciprocal Rank Fusion (RRF): based on rankings



24

Applying rank algorithms to the results from both queries we can sort the final results combining both.

- **RSF- Relative Score Fusion:** Derives each document score by normalizing the vector score and text search score respectively. The highest value becomes 1, the lowest value becomes 0, and others end up in between according to this scale.
- **RRF - Reciprocal Rank Fusion:** Each document is scored according to its position in the results for the given search, starting from the highest score for the top-ranked document and decreasing down the order. The total score is calculated by adding these rank-based scores from the vector and text searches.



Relative Score Fusion (RSF)

Relative Score Fusion (RSF)

1) Normalized text and vector scores

```
      : {  
    "$multiply": [ 1, 1 ],  
    { "$divide": [ 1, 1 ],  
      { "$sum": [ 1, 1 ],  
        { "$exp": {  
          "$multiply": [ -1, 1 ]  
        } } ] } ] } ] } ] }
```

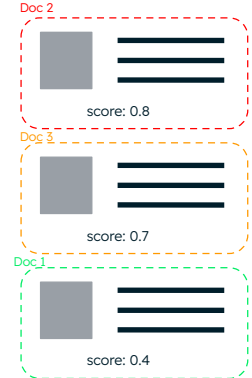
2) Add the scores and re-sort

```
{ "$addFields":  
  {  
    { "$sum": [ "$ts_score", "$vs_score" ] }  
  }  
}  
{"$sort":{"_score": -1 } }
```

Text Results



Vector Results



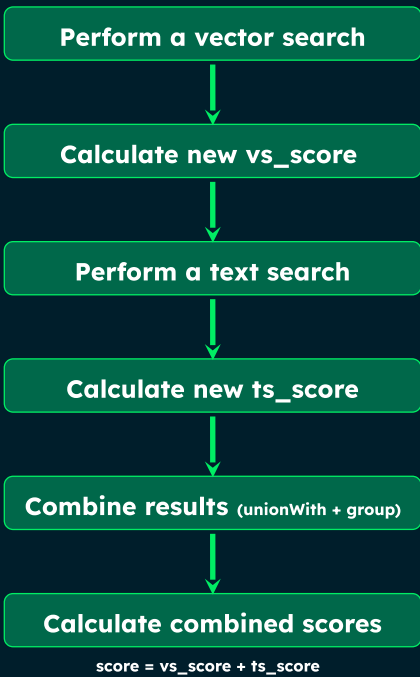
In this Hybrid RS fusion method, we need to recalculate both `ts_score` and `vs_score` using this normalized formula. The normalized formula uses a sigmoid function: https://en.wikipedia.org/wiki/Sigmoid_function

This will allow both vector and text search scores to be in the same scale, and then we can just perform the sum of both to obtain the final score for the hybrid search.

Note: Each type of search can get a different weighting for the final score (for instance: `vs_score` could be calculated using a 0.9 `score_weight` while `ts_score` is calculated using a 0.1 `score_weight`)

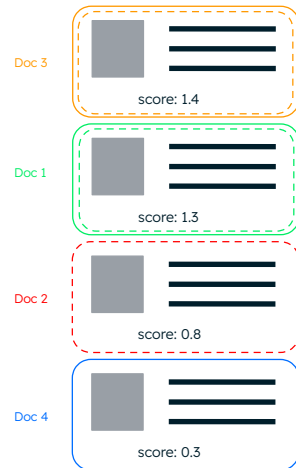
The sum of the weight we associate to the vector score and the text score has to be 1.

$Vs_score_weight + ts_score_weight = 1$



Relative Score Fusion (RSF)

Hybrid Results



Exercise - Hybrid RSF search



Run the hybrid search using the RSF algorithm :

Use the `movie_recommendations_hybrid` project

Review the hybrid search with RSF changes (see `hybridSearchRSF` function)

Execute the search “Hybrid Search RSF” using the same query:

capturing ghosts bill murray

What are the positions of the Ghostbusters movies in the new search results?

29

Use the **movie_recommendations_hybrid** project (available in your lab IDE)

In the **params.py** include the connection string and the API token for our OpenAI azure resource (your instructor will provide you with an OpenAI Key)

In the **recommend.py** script, review the **hybridSearchRSF()** function and the pipeline variable

In a new VSCode terminal execute these commands:

```
cd movie_recommendations_hybrid
python3 -m venv env
source env/bin/activate
pip3 install -r requirements.txt
```

Test in the VSCode Terminal:

```
python3 recommend.py -t "capturing ghosts bill murray" -s "hybridRSF"
```

Exercise - Hybrid RSF (Results)



What movie shall we watch?

This database contains movies with genres of Western, Action and Fantasy

capturing ghosts bill murray
Vector Search | Fulltext Search | Hybrid Search | Hybrid Search RSF | Hybrid Search RRF

Search Results

	Title	Search Score
	Ghostbusters II - Plot: The discovery of a massive river of ectoplasm and a resurgence of spectral activity allows the staff of Ghostbusters to revive the business. - Cast: ['Bill Murray', 'Dan Aykroyd', 'Sigourney Weaver', 'Harold Ramis'] - Genres: ['Comedy', 'Fantasy'] - Rating: 6.5	Score: 0.7677110156964297
	Ghostbusters - Plot: Three unemployed parapsychology professors set up shop as a unique ghost removal service. - Cast: ['Bill Murray', 'Dan Aykroyd', 'Sigourney Weaver', 'Harold Ramis'] - Genres: ['Comedy', 'Fantasy'] - Rating: 7.8	Score: 0.7664419770976534

Hybrid RSF search results

Tested with the following weight values:

- ts_score_factor=0.3
- vs_score_factor=0.7

Ghostbusters movies appear on the 1st and 2nd position in the results

Combined score:
0.76.....

30

You can run this search using the **movies_recommendations_hybrid** python project to see these results:

```
python3 app.py
```

From the browser, use the UI (website) to search:

- Input the search: **capturing ghosts bill murray**
- Use the **Hybrid Search RSF** button

Note: Vector search parameters as well as the score factor variables can also influence the result order



Reciprocal Rank Fusion (RRF)

Reciprocal Rank Fusion (RRF)



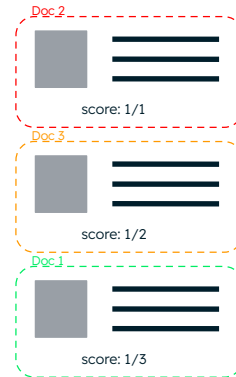
Text Results



Scores are based
on rank

Add the scores
and re-sort

Vector Results



1) Calculate the rank of each search result and use it to get the new score

```
{ "$group": {
  "_id": "null",
  : {"$push": "$$ROOT"}}
},
{ "$unwind": {
  "path": " ",
  "includeArrayIndex":  }
},
{ "$addFields": {
  : {
    "$divide": [ 1,{"$add": [      ,k_penalty, 1]]}
  }}
}}
```

2) Add the scores and re-sort

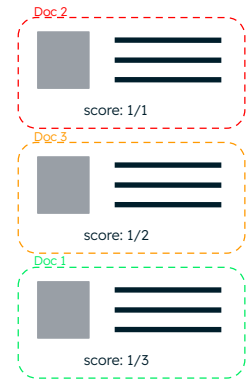
```
{"addFields": {
  "score": {"$add": ["ts_score","vs_score"]} }
}
{"sort":{"score": -1 } }
```

Reciprocal Rank Fusion (RRF)

Text Results



Vector Results



$$\text{Score} = 1 / (\text{rank} + k + 1)$$

33

We calculate the score using an algorithm called reciprocal rank fusion (RRF) with this procedure:

To calculate the Rank fusion score we need to apply a formula to every result in each search stage.

1. Rank the result in the search stage result set.*
2. Assign a weight ("k_penalty"), to each of the two search stages. For best results, we recommend a this penalty to be between 1 and 60.
3. Calculate the per stage value by summing the rank and k values together then computing the reciprocal: $\text{new_score} = 1 / (\text{rank} + k + 1)$
4. Once we have the scores for both search stages, we add them up to get the final score and sort the results by it. The result with the highest fusion score is the most relevant according to this method.

Important: Remember when performing the \$project state the target fields are now in the docs subdocument:

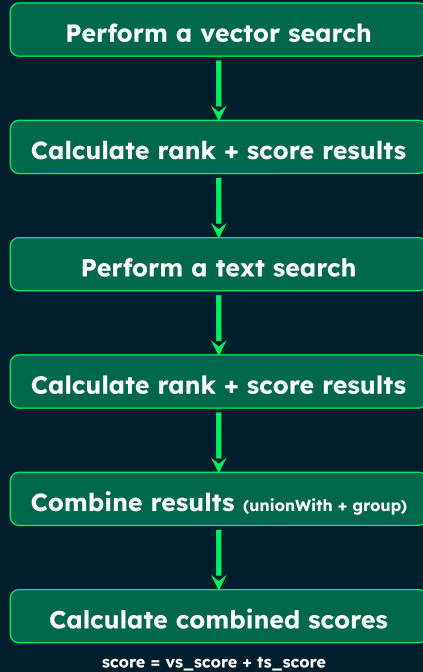
Example:

```
{ "$project": {
  "_id": "$docs._id",
  "title": "$docs.title",
  "plot": "$docs.plot",
  ... }
}
```

*Note: In order to calculate the rank of result, for each search, we need to:

- 1) Perform a \$group operation with "_id": null to list all the documents into an temporary array(docs),
- 2) Perform a \$unwind operation on the docs array stage using "includeArrayIndex": "rank" to obtain the raking of those results

Reciprocal Rank Fusion (RRF)



Hybrid Results

Doc 1



Doc 2



Doc 3



Doc 4



In order to calculate the ranks we will need to work with the array of search results and use `$unwind` and `$group`

Exercise - Hybrid RRF search



Run the hybrid search using the RRF algorithm :

Use the `movie_recommendations_hybrid` project

Review the hybrid search with RSF changes (see `hybridSearchRRF` function)

Execute the search “Hybrid Search RRF” using the same query:

capturing ghosts bill murray

What are the positions of the Ghostbusters movies in the new search results?

36

Use the **movie_recommendations_hybrid** project (available in your lab IDE)

In the **params.py** include the connection string and the API token for our OpenAI azure resource (your instructor will provide you with an OpenAI Key)

In the **recommend.py** script, review the **hybridSearchRRF()** function and the pipeline variable

In a new VSCode terminal execute these commands:

```
cd movie_recommendations_hybrid
python3 -m venv env
source env/bin/activate
pip3 install -r requirements.txt
```

Test in the VSCode Terminal:

```
python3 recommend.py -t "capturing ghosts bill murray" -s "hybridRRF"
```

Exercise - Hybrid RRF (Results)



What movie shall we watch?

This database contains movies with genres of Western, Action and Fantasy

capturing ghosts bill murray
Vector Search | Fulltext Search | Hybrid Search | Hybrid Search RRF | Hybrid Search RRF

Search Results

	Title	Search Score
	Ghostbusters II - Plot: The discovery of a massive river of ectoplasm and a resurgence of spectral activity allows the staff of Ghostbusters to revive the business. - Cast: [Bill Murray, 'Dan Aykroyd', 'Sigourney Weaver', 'Harold Ramis'] - Genres: ['Comedy', 'Fantasy'] - Rating: None	Score: 0.1534090909090909
	Ghostbusters - Plot: Three unemployed parapsychology professors set up shop as a unique ghost removal service. - Cast: [Bill Murray, 'Dan Aykroyd', 'Sigourney Weaver', 'Harold Ramis'] - Genres: ['Comedy', 'Fantasy'] - Rating: None	Score: 0.11929824561403508
	Casper - Plot: A paranormal expert and his daughter bunk in an abandoned house populated by 3 mischievous ghosts and one friendly one. - Cast: [Chauncey Leopardi, 'Spencer Vrooman', 'Malachi Pearson', 'Cathy Moriarty'] - Genres: ['Comedy', 'Family', 'Fantasy'] - Rating: None	Score: 0.11274509803921567
	Beetlejuice Plot: A couple of recently deceased ghosts contract the services of a "bio-exorcist" in order to	

Hybrid RRF search results

Tested with the following weight values:

- ts_k_penalty=10
- vs_k_penalty=10

Ghostbusters movies appear on the 1st and 2nd position in the results

Combined score:
0.153..... / 0.119...

37

You can run this search using the **movies_recommendations_hybrid** python project to see these results:

```
python3 app.py
```

From the browser, use the UI(website) to search:

- Input the search: **capturing ghosts bill murray**
- Use the **Hybrid Search RRF** button

Note: Vector search parameters as well as the penalty variables can also influence the result order

Quiz Time!





#1. What main advantage does MongoDB Hybrid Search have?

A

Uses Lucene indexes

B

Can include additional filters

C

Combines results from both text and vector searches

D

Groups the results to simplify the output

E

Can rank results in different ways

Answer in the next slide.



#1. What main advantage does MongoDB Hybrid Search have?

A

Uses Lucene indexes

B

Can include additional filters

C

Combines results from both text and vector searches

D

Groups the results to simplify the output

E

Can rank results in different ways



#2. What does a fusion algorithm do?

A

Distributes the search scores evenly across all the results

B

Prioritizes the vector search results to focus on the semantic search

C

Reorders the results from both search outputs in a more fair ranking

D

Recalculates the search scores of the results to change the relevance

E

Adjusts the search features defined on the vector search index to avoid incorrect results

Answer in the next slide.



#2. What does a fusion algorithm do?

A

Distributes the search scores evenly across all the results

B

Prioritizes the vector search results to focus on the semantic search

C

Reorders the results from both search outputs in a more fair ranking

D

Recalculates the search scores of the results to change the relevance

E

Adjusts the search features defined on the vector search index to avoid incorrect results



#3. Choose 2 sets of aggregation stages in a hybrid search:

A

\$match
\$facet
\$project

B

\$sort
\$addFields
\$group

C

\$facet
\$unwind
\$limit

D

\$search
\$vectorSearch
\$unionWith

E

\$search
\$vectorSearch
\$set

Answer in the next slide.



#3. Choose 2 sets of aggregation stages in a hybrid search:

A

\$match
\$facet
\$project

B

\$sort
\$addFields
\$group

C

\$facet
\$unwind
\$limit

D

\$search
\$vectorSearch
\$unionWith

E

\$search
\$vectorSearch
\$set

Recap

Atlas Hybrid Search combines Atlas text search ('lexical/full text' search) with Atlas vector search ('semantic search') in an effort to improve the relevance of results

The most common approach is to execute two queries, a text query and a vector query, and then combine the results from each into a new ranked set using RSF or RRF algorithms